AI Agents

Session 1 — Fundamentals, landscape, and your first agent

Juan Cruz-Benito

Master's in Multivariate Statistics · USAL

May 5, 2026 · Salamanca

Who we are

Me

- Who am I?
- Why I teach this class
- How I work day-to-day with agents

Who we are

You

- Physicists, mathematicians, statisticians
- Engineers, computer scientists
- Biologists, psychologists
- Intermediate technical level
- **Your advantage**: you already think in terms of uncertainty, evidence, variance

Initial poll

Raise your hand:

1. **Who has used ChatGPT / Claude / Gemini this week?**
2. **Who has ever called an LLM API from code (Python/R)?**
3. **Who has heard of MCP, function calling, or tool use?**
4. **What do you hope to take away? (one word)**

> The goal is to calibrate the pace, not to judge.

Agenda — Session 1

Block	Content
0	Welcome + poll
1	**Fundamentals: what is an agent, brain, hands, memory, skills**
☕	Break
2	Landscape of frameworks and existing agents
3	Agents you can already use today
4	**"No-code" workshop: your first local agent**
5	Wrap-up and discussion

Logistics

- 💻 **Laptops open**: you'll be installing things
- 📚 **Materials**: this deck + other resources available at https://github.com/cbjuan/2026-agents-usal
- 🙋 **Questions**: whenever you want, no need to wait until the end
- ❓ **Technical issues during the workshop**: we solve them as a group

> If something doesn't work on your laptop during the workshop, **it's perfectly normal**. It's part of the learning.

Materials

github.com/cbjuan/2026-agents-usal

Scan the QR or type the URL. Inside you'll find:

- Both decks (source `.md` + PDF)
- Workshop code templates
- Datasets used in the tasks
- Reading list with links
- Troubleshooting notes

Shared vocabulary

Terms you'll hear repeatedly — this slide stays in the deck for reference:

Term	Meaning
LLM	Large Language Model — GPT, Claude, Gemini, Qwen…
API	Application Programming Interface — calling a service from code
Token	Sub-word unit the LLM reads/writes (~¾ of a word in English)
Prompt	Text input to the LLM; **system prompt** = fixed instructions
Context window	Max tokens the LLM can see at once (4K, 32K, 200K…)
Tool-call	LLM invoking a function (search, read_file, run_sql…)
Inference	One forward pass through the model = cost + latency
RAG	Retrieval-Augmented Generation — fetch docs, add to prompt
MCP	Model Context Protocol — standard to connect tools to LLMs
Fine-tuning	Extra training on specific data to adjust a model's behavior

BLOCK 1

Fundamentals

What (and what isn't) an agent?

The word is everywhere. It means different things:

- In classical AI: an object that **perceives** and **acts upon** an environment
- In products: any app with an LLM
- In venture capital: whatever can raise a Series A

We're not going to invent a canonical definition.

We'll use **five complementary lenses** and pick the useful one for each context.

5 lenses for looking at an agent

Lens	Where it comes from	What it contributes
AIMA (Russell & Norvig)	Classical AI · 1995→	Taxonomy + connection to MDPs/RL
Anthropic	*Building Effective Agents* · 2024	*Workflow* vs *agent*
OpenAI	*Practical guide to building agents* · 2025	Model + Tools + Instructions
LangChain (H. Chase)	*Cognitive architectures*	Autonomy levels 1→6
CoALA (Princeton)	TMLR 2024	Memory + actions + decision

And **Simon Willison**'s operational rule: *"an LLM agent runs tools in a loop to achieve a goal."*

Lens 1 · AIMA (Russell & Norvig)

> *"Anything that can be viewed as perceiving its environment through sensors and acting upon that environment through actuators."*
> — *Artificial Intelligence: A Modern Approach* (1995→2020)

What matters: the definition does not require language, intelligence, or LLMs.
A thermostat is an agent. A human is an agent. A program too.

What does define an agent: having an explicit or implicit **utility function**, and choosing actions that maximize it.

AIMA taxonomy · 5 levels

1. **Simple reflex** — *condition* → *action* rule on the current percept
2. **Model-based reflex** — maintains internal state (partially observable environments)
3. **Goal-based** — picks actions that bring it closer to a goal
4. **Utility-based** — compares states with a utility function
5. **Learning** — adds *learning element + performance element + critic*

An LLM with tools and a loop ≈ hybrid between goal-based and learning-by-context.

Connection with MDPs and RL (the statistician's lens)

An agent can be read as a *policy* over a **Markov Decision Process** (MDP) $\langle S, A, P, R, \gamma \rangle$:

S = states · A = actions · $P(s' \mid s, a)$ = transition probabilities · $R(s, a)$ = reward · $\gamma \in [0, 1]$ = discount factor

- s_t = LLM context (messages + observations)
- a_t = token-stream that produces a *tool-call* or response
- r_t = test that passes, human reward, automated eval
- π_θ = the LLM with its prompt and tools

Three classical **reinforcement learning** (RL) problems reappear identically:

- **Credit assignment** in multi-turn (which step failed?)
- **Exploration** (call a new tool?)
- **Long horizon** — errors compound

Lens 2 · Anthropic — *workflow* vs *agent*

Workflow

Predefined steps in code.

The programmer owns control flow.

```
input → classify → template
      → policy → send
```

Predictable. Auditable. Cheap.

Agent

LLM **decides** which tool to call and when to stop, in a loop.

The model owns control flow.

```
loop: LLM → tool → observation
      → stop?
```

Flexible. Expensive. Non-deterministic.

> **Heuristic**: if you can map the decision tree, **don't build an agent**.

Lens 3 · OpenAI

> *"A system that independently accomplishes tasks on your behalf."*
> — *A practical guide to building agents*, OpenAI · 2025

Three components:

- **Model** — the brain that reasons
- **Tools** — *data*, *action*, *orchestration*
- **Instructions** — policy, format, *guardrails* (safety filters and limits)

Minimal pattern: `while` loop with an exit condition.

When to split into multi-agent: complex logic **or** many overlapping tools.

Lens 4 · LangChain — autonomy levels (synthesis)

Level	Who decides	Example
1. Code only	The human (program)	`df.describe()`
2. LLM call	Human + LLM once	"summarize this text"
3. Chain	Program with several LLMs in fixed order	Classic RAG
4. Router	LLM picks a branch	classifier → specialist
5. State machine	LLM inside a graph with transitions	**LangGraph**
6. Autonomous agent	**LLM in a free loop**	Claude Code, Cursor

> Harrison Chase (LangChain CEO): *"outsource your agentic infrastructure, but **own** your cognitive architecture."*

Lens 5 · CoALA (Cognitive Architectures for Language Agents)

Sumers, Yao, Narasimhan, Griffiths · Princeton · TMLR 2024

An LLM agent has three axes:

Memory (LTM = long-term memory)

- *Working* (in-context)
- Episodic LTM
- Semantic LTM
- Procedural LTM

Actions

- Internal (on memory itself)
- External (on the environment)

Decision

- Planning
- Execution
- Reflection

ReAct, Reflexion, Voyager, Generative Agents… they all fit here.

"An LLM agent runs tools in a loop to achieve a goal."

— Simon Willison · September 2025 (after collecting >200 distinct definitions)

When NOT to build an agent

Three questions (Barry Zhang, Anthropic):

1. **Is the task ambiguous?** If you can map the entire decision tree → **workflow**
2. **Is it worth the cost?** If it's a small, cheap task, an agent is overkill
3. **Is the model capable of the atomic task?** Errors compound multiplicatively

> *"It is the decade of agents, not the year of agents."*
> — Andrej Karpathy · Dwarkesh Podcast, Oct 2025

Errors accumulate multiplicatively. Best to assume it will take time.

Spectrum: chatbot → RAG → workflow → agent

```
Pure chatbot
   └─ One LLM call, no state, no tools

RAG
   └─ LLM + pre-inference retrieval · fixed structure

Workflow
   └─ LLM(s) + tools on code-predefined routes

Agent
   └─ LLM + tools in a loop, decides flow, its own stopping criterion
```

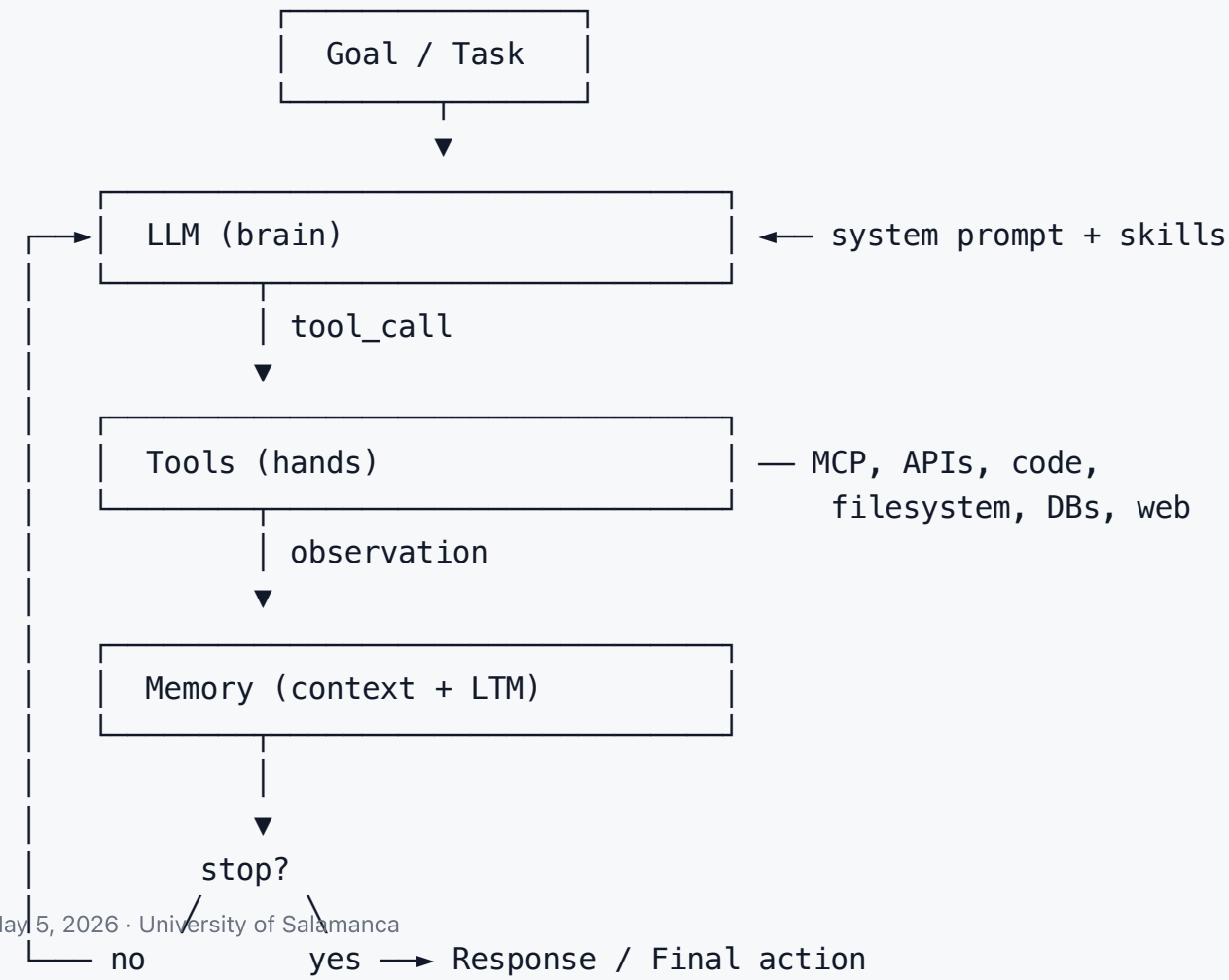
Most "agentic applications" are workflows with an LLM inside.

Discussion question

> To detect **fraud in bank transactions in real time**, would you use an agent or a workflow?
>
> Why?

Discuss with the person next to you. Then we share.

Typical architecture · the agentic loop



The brain · reasoning models in 2026

Frontier — top-tier closed models, accessed via API

- **Claude** (Anthropic) — Opus 4.x, Sonnet 4.x
- **GPT-5.x** (OpenAI)
- **Gemini 3.x** (Google)
- **Grok 4** (xAI)

Open-weights — weights publicly downloadable, runnable on your laptop or server

- **Qwen3 / Qwen3.5 / Qwen3.6** (Alibaba) — Apache 2.0
- **Granite 4.1** (IBM) — Apache 2.0
- **DeepSeek V4 / R1** — large **MoE** (mixture-of-experts) models
- **Phi-4 / Phi-4-mini** (Microsoft) — MIT
- **Gemma 3 / 4** (Google), **SmolLM3** (HuggingFace), **Mistral 3**

Frontier vs open-weights — when which?

Criterion	Frontier (API)	Open-weights (local)
Reasoning quality	★★★★★	★★★ – ★★★★
Complex tool-use	Very reliable	Reliable on Qwen3+, Granite 4.1
Cost per task	$$	$ (electricity)
Latency	Cloud	Depends on hardware
Privacy	Data to the provider	100% local
Reproducibility	Changes without notice	Pinnable versions
Fine-tuning	Limited	Full control

In the workshop we'll use open-weights, not out of ideology, but so it **works without an account and without spending money**.

Lightweight open-weights · top 3 for the workshop

Model	RAM Q4	Tool-use	License	Command
Qwen 3.5 4B	~3.5 GB	Excellent (native)	Apache 2.0	`ollama pull qwen3.5:4b`
Qwen 3.5 9B ⭐	~6.5 GB	Excellent (native)	Apache 2.0	`ollama pull qwen3.5:9b`
Granite 4.1 8B	~5.3 GB	Solid on BFCL	Apache 2.0	`ollama pull granite4.1:8b`
Gemma 4 e2b	~2.5 GB	Native function-calling	Apache 2.0	`ollama pull gemma4:e2b`
Gemma 4 e4b	~4.5 GB	Native function-calling	Apache 2.0	`ollama pull gemma4:e4b`

Q4 = 4-bit quantization (model compressed to ¼ the memory, small quality loss).

BFCL = Berkeley Function Calling Leaderboard, a tool-use benchmark.

Why Qwen 3.5 (not 3.6): Qwen 3.6 only ships at 27B+ — too large for laptops.

> For 32 GB, **Gemma 4 26B** is a fast MoE option: `ollama pull gemma4:26b` (~18 GB).
> Also solid: **Granite 4.1 30B** (~17 GB) and **Nemotron Cascade-2 30B MoE**.

The hands · function calling

How it works in 3 steps:

```
# 1. Describe the tool as a JSON schema
tools = [{
    "name": "descriptive_stats",
    "description": "Compute descriptive statistics of a CSV",
    "parameters": {
        "type": "object",
        "properties": {"path": {"type": "string"}},
        "required": ["path"]
    }
}]

# 2. The LLM responds with a structured object, not text
# {"tool": "descriptive_stats", "args": {"path": "iris.csv"}}

# 3. Your runtime executes the tool and returns its output to the LLM
```

Important: the "magic" lies in the model's *fine-tuning* to emit valid JSON when tools are in the prompt.

MCP · Model Context Protocol

What it is: a universal **JSON-RPC** (remote procedure calls encoded in JSON) protocol between **clients** (Claude Desktop, Cursor, ChatGPT) and **servers** (processes that expose tools, resources, prompts).

Origin: Anthropic, Nov 25, 2024. **Open source** (Linux Foundation since Dec 2025).

Why it matters:

- Moves from **N×M** ad-hoc integrations to **N+M**
- Once you have an MCP server for your DB, **any client** can use it
- De facto standard in 2026

Five primitives: tools, resources, prompts, sampling, roots
Two transports: stdio (local) and Streamable HTTP (remote, OAuth 2.1)

MCP · 2026 adoption state

- **Thousands of MCP servers** published (public registry + private)
- **Tens of millions of monthly downloads** of the SDK (software development kit — the library devs use)
- **Most enterprise teams** report at least one MCP-based agent in production

Native support in:

Claude · ChatGPT (Apps SDK) · Gemini API · Vertex AI · Cursor · Windsurf · Zed · JetBrains AI · OpenAI Agents SDK · Vercel AI SDK

Adjacent ecosystem:

- **A2A** (Google, under Linux Foundation) — agent↔️agent, JSON-RPC
- **ACP** (IBM / BeeAI, now merged into A2A) — REST-native agent interop
- **AG-UI** — frontend↔️agent

Ttools · live example · setup

Instructor demo — same stack you'll install in Block 4: OpenCode + Ollama + a local open-weights model.

Launch OpenCode — model comes from the config (`ollama/gemma4`):

```
ollama launch opencode —-model gemma4
```

Tools · live example · run

Ask in the chat:

> *"List the CSVs in my Downloads folder"*

Under the hood:

1. OpenCode discovers the tools
2. Gemma emits a structured call to the proper tool
3. OpenCode runs it locally → returns the file listing
4. Gemma loops (read each CSV → count rows) → composes the final answer

No cloud. No API key. Same protocols as Claude Desktop and Cursor use.

Memory · four types

Type	What it stores	Examples
Working / context	Active tokens	The LLM itself
Episodic	"What I did last week"	Mem0, Letta, Zep, MemMachine
Semantic	Facts, profiles, KG	Qdrant, pgvector, Neo4j
Procedural	Rules, *skills*, scripts	Claude Skills, AGENTS.md

> CoALA framework: human memory serves as inspiration, not as an exact map.

Context engineering > prompt engineering

> *"The set of strategies for curating and maintaining the optimal set of tokens during inference."*

Mindset shift:

- ❌ From: picking the best 5 sentences of the system prompt
- ✅ To: designing **what information arrives when**

Key techniques:

- **Just-in-time retrieval** — `glob` and `grep` explore on demand
- **Tool-mediated retrieval** — the LLM asks for what it needs
- **Compression** — summary between steps
- **Pruning** — discard the irrelevant
- **Hierarchical loading** — *progressive disclosure* (load detail only when needed)

Skills · *progressive disclosure*

```
---
name: pdf-processing
description: Extract text and tables from PDF files, fill forms.
  Use when the user mentions PDFs or document extraction.
---

# Instructions
1. ...
```

Loads in three levels:

1. **Frontmatter** always in context (~50 tokens)
2. **Body** loads if the skill activates
3. **Sub-files** only if relevant

> Decouples the **size** of the skill from its **cost per token**.

Skills vs MCP vs subagents

Concept	What it provides	When to use it
MCP	Access to tools / data	Connecting to external systems
Skill	Procedure / knowledge	Teaching **how** to do something
Subagent	Process with its own context	Isolate context, parallelize

They're complementary, not mutually exclusive.

Real example: a "statistical analysis" *skill* uses the internal DB's *MCP* and delegates to a "QA" *subagent* to review the output.

Break

BLOCK 2

Framework landscape

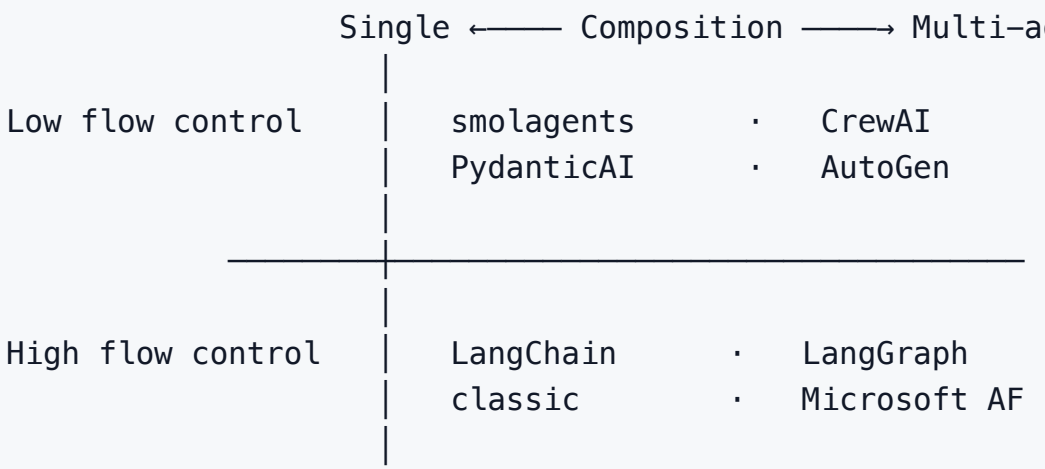
and existing agents

Open source frameworks · 2026 landscape

Framework	Philosophy	Curve
LangGraph	Directed graph with state, checkpoints, HITL	High
CrewAI	Roles + goals + tasks ("crew")	Low
AutoGen / AG2	Multi-agent conversations	Medium
OpenAI Agents SDK	Handoffs between agents	Low
Google ADK	Hierarchical tree, native A2A	Medium
PydanticAI	Extreme type-safety	Medium
Smolagents (HF) ⭐	~1000 LOC, *CodeAgent*, sandbox	**Very low**
Agno	Multi-agent runtime with good DX	Medium
LlamaIndex Agents	Centered on RAG over data	Medium
DSPy	Optimizable prompt programming	High (different paradigm)

HITL = human-in-the-loop · DX = developer experience · LOC = lines of code · SDK = software development kit.

How to choose a framework (simple matrix)



Harrison Chase's rule of thumb:

> *"Outsource your agentic infrastructure, but **own** your cognitive architecture."*

Coding agents · the most mature on the market

Why coding first: tests are verifiable → clear reward signal.

- **Claude Code** (Anthropic, terminal) — current benchmark
- **Cursor / Windsurf** (IDEs)
- **GitHub Copilot Coding Agent**
- **OpenAI Codex** (with Background Computer Use)
- **Devin** (Cognition) — expensive, variable performance depending on benchmark
- **Replit Agent** — long sessions, autonomous
- **OpenCode / Pi** — open source, terminal

> It's the area where agents work best today.

Vibe coding and app builders

They build full-stack apps from natural language:

- **Lovable** · **Bolt** · **v0** (Vercel) · **Same.dev**
- **Manus** · **Leap.new** · **Replit**

Useful for: quick prototypes, landing pages, demos.

Not useful for: real production, refactors, complex systems.

> *"The fastest way to build software is to vibe code your idea, then have someone rewrite it properly."* — the line you hear in SF in 2026.

Computer use · agents that move the mouse

- **Anthropic Computer Use** — receives a screenshot, returns actions
- **ChatGPT Agent Mode** (OpenAI, integrates the old Operator)
- **Google Gemini Computer Use** / Project Mariner
- **Manus AI** — "general **VM** (virtual machine) agent"

Real state: they work, but they're **slow** (seconds per action), **fragile** (UI changes break them), **expensive**.

OSWorld benchmark: performance still clearly below expert human.

Browser agents · the next frontier

- **Perplexity Comet** (free since Oct 2025)
- **ChatGPT Atlas** (Oct 2025, macOS only)
- **Claude for Chrome** (extension)
- **Edge Copilot Mode** · **Opera Aria/Neon** · **Brave Leo**
- **BrowserOS** (open source, Chromium fork with Ollama)

Advantages: session, cookies, login already there.

Risks: prompt injection from any visited page (we'll cover this tomorrow in security).

Open legal case · Amazon vs Perplexity

January 2026: Amazon sues Perplexity for using Comet agents to automate purchases, alleging a bypass of Amazon's **Terms of Service (TOS)**.

Perplexity's position: the agent acts as a human, identifying itself with a standard User-Agent.

Amazon's position: automation at scale violates the terms of service.

> **Question for the group**:
> Is this legitimate TOS defense or resistance to change?
> Who bears responsibility when an agent "clicks"?

BLOCK 3

Agents you can already use today

Recommended stack for a statistics / data profile

Analysis and code

- **Claude Code**, **OpenAI Codex** or **Cursor** for scripts
- **Code Interpreter / Artifacts** for ad-hoc analysis
- **OpenCode + Ollama** local (today's workshop)

Research and synthesis

- **Perplexity** / Perplexity Pro Search
- **NotebookLM** for synthesizing papers
- **Deep Research** (Anthropic, OpenAI, Gemini) or **Elicit** for systematic review

> There's no "correct" stack. What kills you is not trying.

Quick demos

1. **Perplexity Pro Search** — *"robust methods in PCA, 2024-2026 papers"*
2. **NotebookLM** — 3 papers in Spanish → mind map + podcast
3. **Claude / ChatGPT** with artifacts — live CSV analysis
4. **Claude Computer Use** — open Excel, fill out a summary table

What you'll see: the difference between a *chatbot* (saying things) and an *agent* (doing things).

BLOCK 4

Workshop: your first agent without code

Workshop plan

Min	Activity
0–10	**Instructor demo**: Claude Code in action
10–15	Install Ollama (in parallel)
15–20	Download the recommended model
20–25	Install + configure OpenCode
25–55	**Open task**: agent analyzes a CSV and generates a report
55–60	We share results

> **Prerequisites**: ≥ 8 GB RAM, ≥ 15 GB free disk, install permissions.

Instructor demo · Claude Code

Example task:

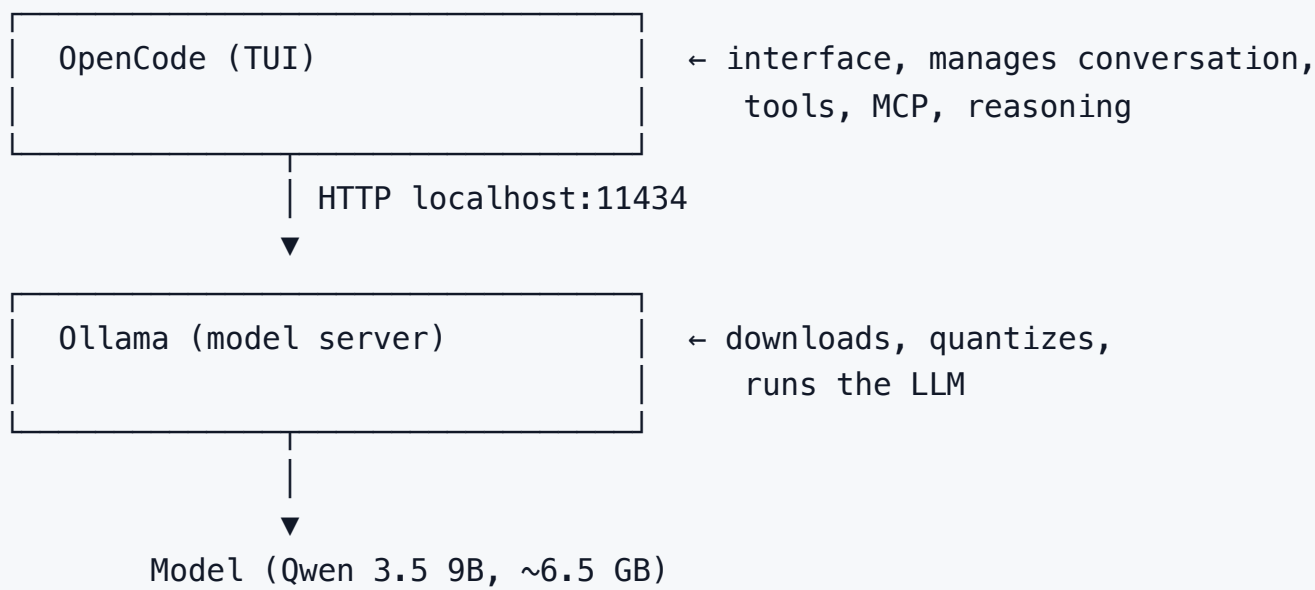
> *"Add NumPy-style docstrings to this module and generate a README based on the tests."*

Show:

- Subagents (Explore, Plan)
- Dynamically loaded skills
- Connected MCP servers (filesystem, git)
- Visible reasoning trace
- When the agent asks for confirmation (**human-in-the-loop / HITL**)

> This is the "ceiling" of what's possible today. You'll build something simpler, but of the same lineage.

Your stack · Ollama + OpenCode



TUI = terminal user interface (text-only, no mouse). Quantize = compress a model to fewer bits per weight.

Both open source. Run offline. No account. Cross-platform.

Step 1 · Install Ollama

```
# macOS / Linux
curl -fsSL https://ollama.com/install.sh | sh

# Windows: download installer from ollama.com

# Verify
ollama --version
```

Check that the server responds:

```
curl http://localhost:11434/api/tags
```

If it returns JSON, you're good.

Step 2 · Download the model

```
# 8 GB RAM
ollama pull qwen3.5:4b          # ~3.5 GB

# 16 GB RAM (recommended)
ollama pull gemma4:e4b          # ~9.4 GB
ollama pull qwen3.5:9b          # ~6.5 GB

# 32 GB RAM (fast MoE)
ollama pull gemma4:26b          # ~18 GB
```

Smoke test:

```
ollama run gemma4 "Hi, what model are you?"
```

> ⚠️ If tools fail later in the workshop, suspect the **default `num_ctx` (4K). You need to raise it to 32K.**

Step 3 · Install OpenCode

```
# Recommended
curl -fsSL https://opencode.ai/install | bash

# Alternatives
npm install -g opencode-ai@latest
brew install sst/tap/opencode

# Verify
opencode --version
```

Create a config file at

`~/.config/opencode/opencode.json` (Linux/macOS)

or `%APPDATA%\opencode\opencode.json` (Windows).

Step 3b · Minimal `opencode.json`

```
{
  "$schema": "https://opencode.ai/config.json",
  "permission": {
    "*": "ask"
  },
  "provider": {
    "ollama": {
      "models": {
        "gemma4": {
          "name": "gemma4",
          "tools": true,
          "reasoning": true,
          "options": { "num_ctx": 32768 }
        },
        "qwen3.5:4b": {
          "name": "qwen3.5:4b",
          "tools": true,
          "reasoning": true,
          "options": { "num_ctx": 32768 }
        }
      },
      "name": "Ollama",
      "npm": "@ai-sdk/openai-compatible",
      "options": {
        "baseURL": "http://127.0.0.1:11434/v1"
      }
    }
  }
}
```

Step 4 · Workshop tasks

Each student picks ONE. Work in pairs if you like.

```
1. Iris analyst
   "Download the Iris dataset from sklearn, compute the
    correlation matrix and save it as heatmap iris_corr.png."

2. Doc generator
   "Generate a Markdown that explains what an agent is,
    citing 3 web sources. Save it as agent_intro.md."

3. CSV explorer
   "Read sales.csv (I'll provide it) and compute mean,
    median, and standard deviation by category."
```

Launch OpenCode:

```
mkdir ~/2026-agents-usal && cd ~/2026-agents-usal
opencode
```

Troubleshooting · if something fails

Symptom	Likely cause	Fix
Model replies with text but **doesn't call tools**	`num_ctx` too low	Raise to 16K-32K in `opencode.json`
`Connection error`	Ollama not running	`ollama serve` in another terminal
Streaming gets cut	Undici timeouts	Update OpenCode to the latest release
Out of memory	Model too large	Switch to `qwen3.5:4b` or `gemma4:e2b`
Tool-call fails with many tools	Hermes-style hallucinates	Reduce tools, raise `num_ctx`
MCP server **SSE** (Server-Sent Events) gives `UnknownError`	Known bug	Use streamable HTTP transport

Pi — minimalist alternative (brief mention)

`pi-mono` (by Mario Zechner / *badlogic*) is a deliberately **anti-feature** *coding agent*:

- **No** MCP, no sub-agents, no permission popups
- Just 4 core tools (read/write/edit/bash)
- Philosophy: *"build CLI tools with READMEs, the agent reads them on demand"*

> Useful as a **pedagogical provocation**: where should complexity live? In the harness or in traditional Unix tools?

Don't install it today. Mentioned for the curious.

Hermes Agent — self-evolving alternative (brief mention)

`hermes-agent` (Nous Research) takes the **opposite bet** to pi:

- **Autonomous skill creation** — generates and refines procedures from experience
- **Persistent memory** across sessions, full-text session search
- **Multi-platform gateway** — Telegram, Discord, Slack, WhatsApp, Signal from one backend
- Runs on anything from a $5 VPS to GPU clusters · **MIT** licence
- Companion repo `hermes-agent-self-evolution` auto-optimises skills + prompts via DSPy + GEPA

> **Pedagogical provocation**: where should the agent's knowledge live? Baked into Unix tools (pi) or evolved at runtime (hermes)?

Don't install it today. Mentioned for the curious.

OpenClaw — batteries-included, local-first (brief mention)

OpenClaw (formerly ClawdBot) sits at the opposite end from pi: *everything bundled, everything local*.

- **Multi-platform** — WhatsApp, Telegram, Discord, Slack, Signal, iMessage
- **50+ integrations** out of the box (Gmail, Notion, HomeKit, browser automation…)
- **Privacy-first** — data stays on your machine unless you explicitly send it out
- **Model-agnostic** — Claude / GPT / Gemini / Llama / Mistral + **Ollama** + LM Studio
- **MIT**, TypeScript · `npm i -g openclaw` · repo openclaw/openclaw

> One axis, three takes: **pi** (nothing included) → **Hermes** (self-evolving) → **OpenClaw** (batteries-included, local).

Don't install it today. Mentioned for the curious.

Wrap-up

What we've seen today

- ✅ Five lenses to define an agent · when NOT to use one
- ✅ Loop: brain + hands + memory + control
- ✅ 2026 models · frontier vs open-weights
- ✅ Function calling · MCP · Skills
- ✅ Frameworks · vertical agents · computer use · browser agents
- ✅ Your first local agent up and running

Tomorrow: how they're built, how they're evaluated, how they break.

Reading for tonight (optional, ~30 min)

Essential:

- Anthropic — *Building Effective Agents* (Dec 2024)
- Anthropic — *Effective Context Engineering* (2025)
- HuggingFace — *Introducing smolagents* (2024)

Supplementary:

- Karpathy on Dwarkesh — *AGI is still a decade away* (Oct 2025)
- Kambhampati et al. — *LLMs Can't Plan, But Can Help Planning* (ICML 2024)
- Simon Willison — *agent-definitions* tag

> If you read only one: Anthropic's **Building Effective Agents**.

Question for tomorrow

> *Which of your tasks would clearly benefit from an **agent** vs. a **rigid workflow**?*
>
> Think it over tonight. We'll discuss at the start tomorrow.

See you tomorrow! 👋

May 6 · 16:00

Building, evaluating, and securing agents

Thank you.